



OUTPUT PARSERS & STRUCTURED OUTPUT

Quick Learning Goal

In this module, we learn how to convert an LLM response into the exact format required by our Python application.

Chat Model



Model Output



String / Structured Data



Python Application

1. Chat Model Output — AIMessage

A LangChain Chat Model normally returns an **AIMessage object**, not just a plain string.

Important Syntax

```
response = llm.invoke("What is Python?")
```

Get the complete object:

```
print(response)
```

Get only generated content:

```
print(response.content)
```

Check type:

```
print(type(response))
```



Program 1 — Understanding AIMessage

```
import os

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4.1-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

response = llm.invoke(
    "What is Python? Explain in one sentence."
)

print(type(response))
print(response)
print(response.content)
```

Remember

```
llm.invoke()
```

↓

AIMessage

```
response.content
```



Generated Content

AIMessage may also contain metadata, token-usage information and other model information.

2. Token Usage and Model Metadata

Useful information can be available inside the response.

Important Syntax

`response.usage_metadata`

`response.response_metadata`

Program 2 — Token Usage

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(  
    model="gpt-4.1-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
response = llm.invoke(  
    "What is Python? Explain in one sentence."  
)
```



```
print(response.usage_metadata["input_tokens"])
print(response.usage_metadata["output_tokens"])
print(response.usage_metadata["total_tokens"])

print(response.response_metadata["model_provider"])
print(response.response_metadata["model_name"])
```

Important

input_tokens → Tokens sent to model

output_tokens → Tokens generated by model

total_tokens → Input + Output

3. What Is Output Processing?

Output processing means converting or extracting model output into the form required by our application.

Model

↓

Model Output

↓

Output Processing

↓

Application-Required Output

An **Output Parser** processes model output.



Important Conclusion

Model generates the answer. Parser processes the answer.

Model $\neq$ Parser

4. StrOutputParser

StrOutputParser is used when our application needs the model's text output as a string.

AIMessage



StrOutputParser



String

Important Syntax

Import:

```
from langchain_core.output_parsers import StrOutputParser
```

Create:

```
parser = StrOutputParser()
```

Parse:

```
result = parser.invoke(response)
```



Program 3 — StrOutputParser

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(
    model="gpt-4.1-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

parser = StrOutputParser()

response = llm.invoke(
    "What is Python? Explain in one sentence."
)

result = parser.invoke(response)

print(result)
```



5. response.content vs StrOutputParser

For a small program:

```
result = response.content
```

is enough.

Using parser:

```
result = parser.invoke(response)
```

is useful when we want a reusable LangChain output-processing component.

response.content



Directly access content

StrOutputParser



Reusable output-processing component

StrOutputParser does **not** generate an answer. It only processes model output.



6. PromptTemplate + Model + Parser

Program 4

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(
    model="gpt-4.1-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = PromptTemplate.from_template(
    "Explain {topic} in very simple English with one example."
)

parser = StrOutputParser()

topic = input("Enter topic: ")

prompt = template.invoke(
    {
```



```
"topic": topic
}
)

response = llm.invoke(prompt)

result = parser.invoke(response)

print(result)
```

Flow

```
User Input
↓
PromptTemplate
↓
Chat Model
↓
AIMessage
↓
StrOutputParser
↓
String
```



7. JSON Output

JSON is a common structured data format.

Example:

```
{  
    "name": "Python",  
    "category": "Programming Language",  
    "difficulty": "Beginner Friendly"  
}
```

JSON and Python dict may look similar, but they are different concepts.

JSON → Data format

Python dict → Python data type

Program 5 — Asking LLM for JSON

```
import os  
  
from langchain_openai import ChatOpenAI  
  
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)  
  
response = llm.invoke(  
    ""
```



Give information about Python.

Return only JSON with these fields:

name

category

difficulty

"""

)

print(response.content)

This is a basic **prompt-based JSON approach**.

For stronger application structure, we use a **schema** and structured output.

8. What Is Structured Output?

Normal free-form output:

Ravi is 22 years old and learning Python.

Structured output:

name = Ravi

age = 22

course = Python



Structured output means returning information according to a predefined structure/schema.

Natural Language



LLM + Schema



Structured Data

9. What Is a Schema?

A **schema** defines the fields and data types required by our application.

Example:

name → str

age → int

course → str

In Python:

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    course: str
```



10. Pydantic

Pydantic helps us:

Define Data Structure

+

Validate Data

Important Syntax

```
from pydantic import BaseModel
```

Create schema:

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    course: str
```



11. Basic Pydantic Program

Program 6

```
from pydantic import BaseModel
```

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    course: str
```

```
student = Student(
```

```
    name="Ravi",
```

```
    age=22,
```

```
    course="Python"
```

```
)
```

```
print(student)
```

```
print(student.name)
```

```
print(student.age)
```

```
print(student.course)
```

Important

Student → Pydantic Model / Schema

student → Pydantic Object



Access fields using:

student.name

student.age

student.course

12. Data Types in Pydantic

Example:

```
class Employee(BaseModel):
```

```
    name: str
```

```
    experience: int
```

```
    salary: float
```

```
    active: bool
```

Common types:

str → Text

int → Integer

float → Decimal number

bool → True / False

list[str] → List of strings



13. Required Fields

Consider:

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    course: str
```

All three fields are required.

If course is missing:

```
student = Student(
```

```
    name="Ravi",
```

```
    age=22
```

```
)
```

Pydantic raises a:

ValidationError



Program 7 — Handling ValidationError

```
from pydantic import BaseModel, ValidationError
```

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    course: str
```

```
try:
```

```
    student = Student(
```

```
        name="Ravi",
```

```
        age=22
```

```
    )
```

```
    print(student)
```

```
except ValidationError as e:
```

```
    print("Validation Error:")
```

```
    print(e)
```



14. Optional Fields

If a value may be unavailable, make it optional.

Important Syntax

email: str | None = None

Example:

```
from pydantic import BaseModel
```

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
    email: str | None = None
```

```
student = Student(
```

```
    name="Ravi",
```

```
    age=22
```

```
)
```

```
print(student)
```

Output concept:

name='Ravi' age=22 email=None

Remember

name: str

Required.



email: str | None = None

Optional with default None.

15. Field()

Field() allows us to provide additional information about a field.

Important Syntax

from pydantic import BaseModel, Field

Example:

```
class Product(BaseModel):
```

```
    name: str = Field(  
        description="Name of the product"  
    )
```

```
    category: str = Field(  
        description="Category of the product"  
    )
```

```
    price: float = Field(  
        description="Price of the product"  
    )
```

```
    in_stock: bool = Field(  

```



```
description="Whether the product is available"
```

```
)
```

Useful formula:

Field Name

+

Data Type

+

Description

Descriptions are especially useful when the schema is given to an LLM.

16. Default Value with Field()

Important Syntax

```
in_stock: bool | None = Field(  
    default=False,  
    description="Whether the product is available"  
)
```

Example:

```
p = Product(  
    name="Mobile",  
    category="smartphone",  
    price=123.45  
)
```

If no value is supplied:



in_stock=False

17. Pydantic Object → Dictionary / JSON

Three very important operations:

product.name

Gets one field.

product.model_dump()

Converts the Pydantic object into a Python dictionary.

product.model_dump_json()

Converts it into a JSON string.

Program 8

```
from pydantic import BaseModel, Field
```

```
class Product(BaseModel):
```

```
    name: str = Field(  
        description="Name of the product"  
    )
```

```
    category: str = Field(  
        description="Category of the product"  
    )
```



```
price: float = Field(
    description="Price of the product"
)

in_stock: bool | None = Field(
    default=False,
    description="Whether the product is available"
)

product = Product(
    name="Laptop",
    category="Electronics",
    price=65000,
    in_stock=True
)

print(product)

data = product.model_dump()

print(data)
print(data["name"])
print(data["category"])
```



```
print(data["price"])
```

```
json_data = product.model_dump_json()
```

```
print(json_data)
```

Quick Comparison

Syntax	Result
product	Pydantic object
product.name	Individual field
product.model_dump()	Python dictionary
product.model_dump_json()	JSON string

18. LangChain with_structured_output()

This is one of the most important syntaxes of Module 6.

Important Syntax

```
structured_llm = llm.with_structured_output(Student)
```

Then:

```
result = structured_llm.invoke(text)
```

With a Pydantic schema, we can work with the structured result as a Pydantic object.



19. Basic Structured Output Program

Program 9 — Student Extractor

```
import os

from langchain_openai import ChatOpenAI
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    age: int
    course: str

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(Student)

result = structured_llm.invoke(
    "Ravi is 22 years old and he is learning Python."
)
```



```
print(result)
```

```
print(type(result))
```

```
print(result.name)
```

```
print(result.age)
```

```
print(result.course)
```

Flow

Natural Language

↓

Chat Model

+

Student Schema

↓

Student Object

Normal vs Structured

```
llm.invoke(...)
```

↓

AIMessage

```
structured_llm.invoke(...)
```

↓

Pydantic Object



20. Complete Employee Extractor

Program 10

```
import os

from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class Employee(BaseModel):

    name: str = Field(
        description="Name of the employee"
    )

    technology: str = Field(
        description="Primary technology of the employee"
    )

    experience: int = Field(
        description="Years of professional experience"
    )

llm = ChatOpenAI(
    model="gpt-4o-mini",
```



```
api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(Employee)

text = input(
    "Enter employee information: "
)

employee = structured_llm.invoke(text)

print("Name    :", employee.name)
print("Technology :", employee.technology)
print("Experience :", employee.experience)

Sample input:
Ramesh is a Java developer with 6 years of experience.
```



21. Structured Output with Missing Information

Do not force the model to invent unavailable information.

If a value may be missing:

experience: int | None = None

Good extraction instruction:

Use only supplied information.

Do not invent information.

If optional information is unavailable, use null.

Important

Correct Structure $\neq$ Guaranteed Correct Facts

Good structured-output applications require:

Good Schema

+

Clear Prompt

+

Optional Fields

+

Validation

+

Application Checks



22. include_raw

By default:

```
structured_llm = llm.with_structured_output(Student)
```

means:

```
include_raw=False
```

When successful, we directly receive the parsed object.

For debugging, use:

Important Syntax

```
structured_llm = llm.with_structured_output(  
    Student,  
    include_raw=True  
)
```

Now the result contains:

```
result["raw"]
```

```
result["parsed"]
```

```
result["parsing_error"]
```

Meaning

```
raw
```

↓

Original model response

```
parsed
```

↓



Parsed Pydantic object

parsing_error

↓

Parsing error, if any

23. include_raw=True Program

Program 11

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Employee(BaseModel):
```

```
    name: str = Field(  
        description="Name of the employee"  
    )
```

```
    technology: str = Field(  
        description="Primary technology"  
    )
```



```
experience: int = Field(
    description="Professional experience in years"
)

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(
    Employee,
    include_raw=True
)

result = structured_llm.invoke(
    "Ramesh is a Java developer with 6 years of experience."
)

print("\n----- RAW -----")
print(result["raw"])

print("\n----- PARSED -----")
print(result["parsed"])
```



```
print("\n----- PARSING ERROR -----")  
print(result["parsing_error"])
```

Remember

If parsing succeeds:

result["parsing_error"] is None

24. Reliable Student Extractor

Program 12

```
import os  
  
from pydantic import BaseModel, Field  
from langchain_openai import ChatOpenAI  
  
class Student(BaseModel):  
  
    name: str = Field(  
        description="Name of the student"  
    )  
  
    age: int | None = Field(  
        default=None,  
        description="Student age if explicitly provided"
```



```
)

technology: str | None = Field(
    default=None,
    description="Technology the student is learning"
)

experience: int | None = Field(
    default=None,
    description="Experience in years if explicitly provided"
)

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(
    Student,
    include_raw=True
)

text = input("Enter information: ")
```



```
prompt = f"""
```

```
Extract student information.
```

```
Rules:
```

1. Use only the supplied text.
2. Do not invent information.
3. If optional information is unavailable, use null.

```
Text:
```

```
{text}
```

```
"""
```

```
result = structured_llm.invoke(prompt)
```

```
if result["parsing_error"] is None:
```

```
    student = result["parsed"]
```

```
    print("Parsing Successful")
```

```
    print("Name    :", student.name)
```

```
    print("Age     :", student.age)
```



```
print("Technology :", student.technology)
print("Experience :", student.experience)
```

else:

```
print("Parsing Failed")

print(result["parsing_error"])
print(result["raw"])
```

Key Syntax

```
if result["parsing_error"] is None:
    then:
        student = result["parsed"]
```

25. Restricting Values with Literal

Sometimes an application should accept only specific values.

Example:

positive

negative

neutral

Use:

Important Syntax

```
from typing import Literal
```



sentiment: Literal[

"positive",

"negative",

"neutral"

]

This helps us define predictable categories.

26. Numeric Validation with Field()

Example:

confidence: float = Field(

ge=0,

le=1,

description="Confidence score between 0 and 1"

)

Meaning:

ge → greater than or equal to

le → less than or equal to

Therefore:

$0 \leq \text{confidence} \leq 1$



27. Sentiment Analyzer

Program 13

```
import os

from typing import Literal

from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class SentimentResult(BaseModel):

    sentiment: Literal[
        "positive",
        "negative",
        "neutral"
    ] = Field(
        description="Overall sentiment of the text"
    )

    confidence: float = Field(
        ge=0,
        le=1,
        description="Confidence score between 0 and 1"
```



```
)

reason: str = Field(
    description="Very short reason for the sentiment"
)

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(
    SentimentResult
)

text = input(
    "Enter customer feedback: "
)

result = structured_llm.invoke(text)

print("\nSENTIMENT ANALYSIS")
print("-----")
```



```
print("Sentiment :", result.sentiment)
print("Confidence :", result.confidence)
print("Reason   :", result.reason)
```

Structured output can directly participate in Python logic:

```
if result.sentiment == "negative":
    print("Escalate to support team")
else:
    print("No escalation required")
```



28. Support Ticket Classifier

This combines:

Literal

+

Field()

+

Numeric Constraints

+

Structured Output

Program 14

```
import os
```

```
from typing import Literal
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class SupportTicket(BaseModel):
```

```
    category: Literal[
```

```
        "billing",
```

```
        "technical",
```

```
        "account",
```



"other"

```
] = Field(  
    description="Main category of the customer issue"  
)
```

```
priority: Literal(  
    "low",  
    "medium",  
    "high"  
) = Field(  
    description="Priority level of the issue"  
)
```

```
severity: int = Field(  
    ge=1,  
    le=5,  
    description="Severity from 1 to 5"  
)
```

```
summary: str = Field(  
    description="Very short summary of the customer issue"  
)
```



```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)  
  
structured_llm = llm.with_structured_output(  
    SupportTicket  
)  
  
text = input(  
    "Enter customer issue: "  
)  
  
result = structured_llm.invoke(text)  
  
print("\nSUPPORT TICKET")  
print("-----")  
  
print("Category :", result.category)  
print("Priority :", result.priority)  
print("Severity :", result.severity)  
print("Summary :", result.summary)  
Sample inputs:
```



My payment was deducted twice from my credit card.

Please refund the extra amount.

or:

I forgot my password and cannot login to my account.

29. Most Important Syntax Cheat Sheet

Get normal model response

```
response = llm.invoke(prompt)
```

Get generated content

```
response.content
```

Token usage

```
response.usage_metadata
```

String parser

```
parser = StrOutputParser()
```

```
result = parser.invoke(response)
```

Pydantic model

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```



Optional field

email: str | None = None

Field description

```
name: str = Field(  
    description="Name of the student"  
)
```

Default value

```
active: bool = Field(  
    default=False  
)
```

Pydantic object → dict

```
obj.model_dump()
```

Pydantic object → JSON

```
obj.model_dump_json()
```

Structured output

```
structured_llm = llm.with_structured_output(Student)
```



Execute structured model

result = structured_llm.invoke(text)

Access field

result.name

Include original response

structured_llm = llm.with_structured_output(

 Student,

 include_raw=True

)

Raw model output

result["raw"]

Parsed object

result["parsed"]

Parsing error

result["parsing_error"]

Check parsing

if result["parsing_error"] is None:



```
data = result["parsed"]
```

```
# Restricted values
```

```
status: Literal["open", "closed"]
```

```
# Numeric constraint
```

```
score: float = Field(
```

```
    ge=0,
```

```
    le=1
```

```
)
```

30. Module 6 Master Comparison

When the application needs normal text

Chat Model

↓

AIMessage

↓

StrOutputParser

↓

String

Use:

```
parser = StrOutputParser()
```



When the application needs structured data

Chat Model

+

Pydantic Schema

↓

Structured Output

↓

Pydantic Object

↓

Python Application

Use:

```
structured_llm = llm.with_structured_output(  
    MySchema  
)
```

31. Important Student Points

AIMessage is a message object, not simply a string.

response.content gives generated content.

StrOutputParser processes text output; it does not generate the answer.

Schema defines required fields and types.

Pydantic defines and validates data models.

Required field must have a value.

Optional field can use None.



Field() gives descriptions, defaults and validation rules.

model_dump() gives a dictionary.

model_dump_json() gives JSON text.

with_structured_output() returns data according to a schema.

include_raw=True gives raw, parsed, and parsing_error.

Literal restricts a value to fixed choices.

Structured output gives predictable structure, but it does **not** automatically guarantee that every generated fact is correct.





Ultra-Short Coding Cheat Sheet

Concept	Meaning	Key Coding Line
Normal LLM Call	Send input to Chat Model	<code>response = llm.invoke(prompt)</code>
AIMessage Content	Get generated text	<code>response.content</code>
Input Tokens	Tokens sent to model	<code>response.usage_metadata["input_tokens"]</code>
Output Tokens	Tokens generated	<code>response.usage_metadata["output_tokens"]</code>
Total Tokens	Input + Output tokens	<code>response.usage_metadata["total_tokens"]</code>
StrOutputParser	Convert model output to text/string	<code>parser = StrOutputParser()</code>
Parse Output	Process AIMessage	<code>result = parser.invoke(response)</code>
Pydantic Schema	Define expected data structure	<code>class Student(BaseModel):</code>
Field Type	Specify expected datatype	<code>age: int</code>
Optional Field	Value may be unavailable	<code>email: str None = None</code>
Field Description	Clearly describe a field	<code>name: str = Field(description="Student name")</code>
Default Value	Use value when not supplied	<code>active: bool = Field(default=False)</code>



Concept	Meaning	Key Coding Line
Validation Error	Catch invalid Pydantic data	except ValidationError as e:
Access Field	Read value from Pydantic object	student.name
Object → Dictionary	Convert Pydantic object to dict	student.model_dump()
Object → JSON	Convert Pydantic object to JSON string	student.model_dump_json()
Structured Output	Make LLM follow a schema	structured_llm = llm.with_structured_output(Student)
Structured Invocation	Get structured result	result = structured_llm.invoke(text)
Raw Output Required	Include original model response	include_raw=True
Raw Response	Original model response	result["raw"]
Parsed Response	Successfully structured object	result["parsed"]
Parsing Error	Error while creating structured result	result["parsing_error"]
Check Parsing Success	Verify no parsing error	if result["parsing_error"] is None:
Fixed Allowed Values	Restrict output choices	sentiment: Literal["positive", "negative", "neutral"]



Concept	Meaning	Key Coding Line
Minimum Constraint	Minimum allowed value	ge=0
Maximum Constraint	Maximum allowed value	le=1

Three Most Important Flows

AIMessage → response.content → Text

AIMessage → StrOutputParser → String

LLM + Pydantic Schema → with_structured_output() → Pydantic Object

Final Memory

```
response = llm.invoke(prompt)
```

```
result = parser.invoke(response)
```

```
structured_llm = llm.with_structured_output(Student)
```

```
result = structured_llm.invoke(text)
```

```
result["raw"]
```

```
result["parsed"]
```

```
result["parsing_error"]
```